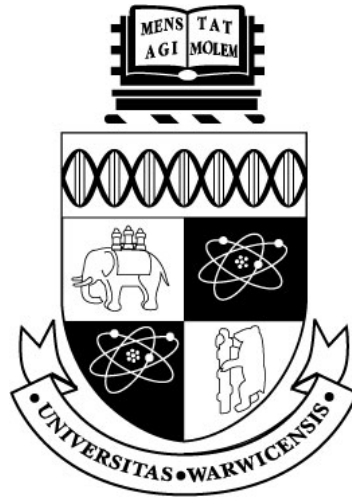




# IMPROVING THE READABILITY AND RUNTIME INSPECTION OF LARGE VISUAL BEHAVIOUR TREES IN GODOT 4.6

by

[AUTHOR NAME]

A thesis submitted in partial fulfilment  
of the requirements for the degree of

MASTER OF SCIENCE IN GAMES  
ENGINEERING

UNIVERSITY OF WARWICK

WMG, University of Warwick

September 2026

# CONTENTS

|                                                                 |             |
|-----------------------------------------------------------------|-------------|
| <b>Contents</b>                                                 | <b>ii</b>   |
| <b>List of figures</b>                                          | <b>v</b>    |
| <b>List of tables</b>                                           | <b>vi</b>   |
| <b>Acknowledgements</b>                                         | <b>vii</b>  |
| <b>Declaration</b>                                              | <b>viii</b> |
| <b>Abstract</b>                                                 | <b>ix</b>   |
| <b>Abbreviations</b>                                            | <b>x</b>    |
| <b>1 Introduction</b>                                           | <b>1</b>    |
| 1.1 Background and context . . . . .                            | 1           |
| 1.2 Problem statement and research gap . . . . .                | 2           |
| 1.3 Research aim and questions . . . . .                        | 2           |
| 1.4 Objectives and success criteria . . . . .                   | 3           |
| 1.5 Contributions . . . . .                                     | 3           |
| 1.6 Dissertation structure . . . . .                            | 4           |
| <b>2 Literature Review</b>                                      | <b>5</b>    |
| 2.1 Behaviour trees as executable hierarchies . . . . .         | 5           |
| 2.2 Node–link trees and scalability . . . . .                   | 5           |
| 2.3 Focus, context and overview . . . . .                       | 6           |
| 2.4 Complexity management and mental-map preservation . . . . . | 7           |
| 2.5 Runtime explanation and failure localisation . . . . .      | 7           |
| 2.6 Research gap and design implications . . . . .              | 8           |
| <b>3 Research Methodology</b>                                   | <b>9</b>    |
| 3.1 Research strategy . . . . .                                 | 9           |
| 3.2 Experimental factors and datasets . . . . .                 | 9           |
| 3.3 Software correctness and integration evaluation . . . . .   | 10          |
| 3.4 Visual validation . . . . .                                 | 10          |

|          |                                                      |           |
|----------|------------------------------------------------------|-----------|
| 3.5      | Human-study design . . . . .                         | 11        |
| 3.6      | Validity and reproducibility . . . . .               | 11        |
| <b>4</b> | <b>System Design and Implementation</b>              | <b>12</b> |
| 4.1      | Architecture and project boundary . . . . .          | 12        |
| 4.2      | Resource model and validation . . . . .              | 13        |
| 4.3      | Runtime semantics . . . . .                          | 13        |
| 4.4      | Editor authoring workflow . . . . .                  | 13        |
| 4.5      | Live debugging and blackboard inspection . . . . .   | 15        |
| 4.6      | Large-tree display mechanisms . . . . .              | 15        |
| 4.6.1    | Information-density control . . . . .                | 15        |
| 4.6.2    | Focus and structural reduction . . . . .             | 16        |
| 4.6.3    | Navigation and overview . . . . .                    | 16        |
| 4.6.4    | Edge and execution representations . . . . .         | 16        |
| 4.7      | Runtime topology cache . . . . .                     | 17        |
| 4.8      | Demonstration game . . . . .                         | 17        |
| <b>5</b> | <b>Results and Analysis</b>                          | <b>18</b> |
| 5.1      | Functional correctness . . . . .                     | 18        |
| 5.2      | Display-density and context results . . . . .        | 19        |
| 5.3      | Runtime-path and overview results . . . . .          | 19        |
| 5.4      | Visual-regression findings . . . . .                 | 20        |
| 5.5      | Runtime-cache results . . . . .                      | 21        |
| 5.6      | Summary against research questions . . . . .         | 22        |
| <b>6</b> | <b>Discussion</b>                                    | <b>23</b> |
| 6.1      | A complete system as a research instrument . . . . . | 23        |
| 6.2      | Interpreting the visualisation results . . . . .     | 23        |
| 6.3      | Live debugging as a readability mechanism . . . . .  | 24        |
| 6.4      | Runtime-cache trade-offs . . . . .                   | 25        |
| 6.5      | Reliability and packaging . . . . .                  | 25        |
| 6.6      | Threats to validity . . . . .                        | 25        |
| 6.6.1    | Construct validity . . . . .                         | 25        |
| 6.6.2    | Internal validity . . . . .                          | 26        |
| 6.6.3    | External validity . . . . .                          | 26        |
| 6.6.4    | Researcher and participant validity . . . . .        | 26        |
| 6.7      | Practical implications . . . . .                     | 26        |
| <b>7</b> | <b>Conclusions</b>                                   | <b>27</b> |
| 7.1      | Summary . . . . .                                    | 27        |
| 7.2      | Contributions . . . . .                              | 27        |
| 7.3      | Limitations . . . . .                                | 28        |

## CONTENTS

iv

|     |                                        |           |
|-----|----------------------------------------|-----------|
| 7.4 | Future work . . . . .                  | 28        |
| 7.5 | Final statement . . . . .              | 28        |
|     | <b>Bibliography</b>                    | <b>29</b> |
| A   | <b>Feature Reference</b>               | <b>31</b> |
| B   | <b>Reproduction and Evidence Index</b> | <b>33</b> |
| C   | <b>Planned Participant Study</b>       | <b>34</b> |
| D   | <b>Draft Completion Checklist</b>      | <b>35</b> |

# LIST OF FIGURES

|     |                                                                                                                                    |    |
|-----|------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.1 | Implemented plugin architecture and data flow between the Godot editor, behaviour-tree resource, runtime runner and actor. . . . . | 12 |
| 4.2 | Editor-only live debugging with the active path and a failure explanation attached to the relevant branch. . . . .                 | 15 |
| 4.3 | Full-detail baseline and compact-card presentation of the same behaviour tree.                                                     | 16 |
| 5.1 | Measured geometric and salience effects for principal methods on the 364-node fixture. . . . .                                     | 20 |
| 5.2 | Examples of focus+context, structural reduction and salience-based navigation in the implemented editor. . . . .                   | 21 |
| 5.3 | Runtime tick scaling with the safe topology cache enabled and disabled. . . .                                                      | 21 |

# LIST OF TABLES

|     |                                                                  |    |
|-----|------------------------------------------------------------------|----|
| 4.1 | Implemented runtime node semantics. . . . .                      | 14 |
| 5.1 | Final core automated regression. . . . .                         | 18 |
| 5.2 | Principal display measurements on fixed generated trees. . . . . | 19 |
| 5.3 | Active-branch dimming and enhanced-minimap measurements. . . . . | 20 |
| 5.4 | Median runtime profile by tree size and runner count. . . . .    | 22 |
| A.1 | Independent display features in the final editor. . . . .        | 31 |

# ACKNOWLEDGEMENTS

This draft reserves space to acknowledge the project supervisor, participants in the planned user study, and anyone who reviewed the software or dissertation. The final wording should be completed by the author before submission.

# DECLARATION

This thesis is submitted to the University of Warwick in support of my application for the degree of Master of Science in Games Engineering. It has been composed by myself and has not been submitted in any previous application for any degree. The software, test data and analysis presented in this dissertation were produced as part of the project unless a source is explicitly cited. No participant results are reported: the human-comparison protocol is prepared, but recruitment and data collection remain future work. This statement must be reviewed and personalised by the author before submission, including any required declaration concerning the use of generative artificial intelligence.

# ABSTRACT

Visual behaviour trees are widely used to author game-agent decisions, but conventional node–link editors become difficult to inspect as trees grow and as runtime state changes. This dissertation presents the design and evaluation of a Godot 4.6 editor plugin that combines a complete executable behaviour-tree system with independently switchable techniques for large-tree readability and live debugging. The runtime supports ordered, reactive, random, parallel, repeating and timed control flow; actor method invocation; typed blackboard data; decorators; and editor-only execution inspection. The editor implements compact cards, semantic zoom, fisheye focus+context, subtree collapse and focus, search highlighting, overview+detail, stable layout, alternative edge routing, active-path emphasis and failure annotation.

Evaluation used deterministic 31-, 121- and 364-node trees, a complex 2D guard scenario, automated semantic and GUI tests, and real-GPU visual regression. All 447 core assertions passed, alongside 21 controlled-study visual assertions, 511 runtime-profile assertions and 53 clean-package checks. On the 364-node tree, compact cards reduced measured card area by 55.9%, subtree focus reduced visible nodes by 97.5%, collapse by 99.5%, search dimmed 99.7% of non-target nodes, and the minimap retained 100% node coverage. A safe topology cache reduced median tick cost by 15.98–26.46% for 31 nodes, 52.61–56.02% for 121 nodes and 76.65–78.00% for 364 nodes. These results establish implementation correctness and geometric effects, but do not by themselves prove improved human readability. A counterbalanced participant study is therefore specified as the principal next step.

# ABBREVIATIONS

**AI** Artificial intelligence

**BT** Behaviour tree

**DOI** Degree of interest

**GPU** Graphics processing unit

**GUI** Graphical user interface

**NPC** Non-player character

**RQ** Research question

**UE** Unreal Engine

# 1 INTRODUCTION

## 1.1 BACKGROUND AND CONTEXT

Modern game agents must choose between actions such as patrolling, pursuing a target, attacking, retreating and recovering. Hard-coding these decisions as a single procedural script can work for a small prototype, but increasing the number of states and interruptions makes control flow difficult to inspect and modify. A behaviour tree (BT) represents decision logic as a hierarchy of control-flow and leaf nodes. Composite nodes determine how children are selected, while conditions and actions connect the abstract decision structure to game state and actor code. The resulting separation is attractive for game development because an agent’s high-level policy can be edited without rewriting every action implementation (Colledanchise and Ögren, 2018).

Commercial engines demonstrate the practical value of visual behaviour authoring. Unreal Engine combines a node–link tree, decorators and a blackboard, and displays execution state in the editor (Epic Games, 2026). Godot 4.6 provides general editor-extension and graph controls, notably `EditorPlugin`, `GraphNode` and `GraphEdit`, but does not prescribe a complete visual behaviour-tree workflow (Godot Engine contributors, 2026b,a). This creates an engineering opportunity and a research problem. A plugin must first execute trees correctly and integrate with arbitrary actors; once the tree is useful, its visual representation must remain understandable as the number of nodes and runtime annotations increases.

Conventional node–link trees communicate parent–child topology directly, but their use of space deteriorates with breadth and depth. A behaviour node may also display parameters, descriptions, decorators, blackboard keys and execution state, making each card larger than a label-only hierarchy. When dozens or hundreds of cards are placed on a finite editor canvas, developers must repeatedly zoom, pan and search. The active execution path can be visually lost among inactive branches. This is not only an aesthetic issue: the representation is the primary instrument through which designers locate tasks, understand ordering, edit conditions and diagnose failure.

The project therefore treats a functional Godot behaviour-tree plugin as the experimental platform for studying large-tree presentation. The implementation follows a recognisable top-to-bottom visual language with ordered children executing from left to right, while adding switchable focus+context, complexity-management, navigation and runtime-explanation techniques. Each technique can be disabled independently so that a visual

fault cannot permanently destabilise the editor and so that baseline comparisons remain possible.

## 1.2 PROBLEM STATEMENT AND RESEARCH GAP

Research has produced many techniques for tree and graph visualisation. Fisheye and focus+context views allocate more display space to items near a focus (Furnas, 1986; Lamping et al., 1995); overview+detail preserves a global reference view (Cockburn et al., 2008); multi-column presentation assists browsing in trees with large fan-outs (Song et al., 2010); and incremental expand–collapse operations reduce graph complexity while attempting to preserve the mental map (Dogrusoz et al., 2018; Misue et al., 1995). Large layered-graph research further shows that path-following performance depends on representation and task (Bae and Watson, 2011).

However, these findings do not directly answer how a game-development behaviour-tree editor should combine such mechanisms. Behaviour trees differ from generic file hierarchies because sibling order has execution semantics, leaf nodes invoke code, decorators can prevent execution, and the graph changes state every runtime tick. A useful solution must preserve semantic order, support editing and undo, expose blackboard state, and explain failures without adding a debugging overlay to the game itself. It must also distinguish measurable geometric improvements from claims about human comprehension. Existing studies motivate individual techniques, but an integrated and reproducibly evaluated Godot implementation is still needed.

## 1.3 RESEARCH AIM AND QUESTIONS

The aim of this dissertation is to design, implement and evaluate a Godot 4.6 visual behaviour-tree plugin that supports real non-player character (NPC) execution while providing robust, switchable mechanisms for inspecting large trees.

The study addresses three research questions:

- RQ1:** Can a Godot editor plugin provide the core authoring, runtime, blackboard, decorator and live-debug capabilities required to control a non-trivial 2D game agent?
- RQ2:** How do independently switchable visualisation techniques affect measurable properties of 31-, 121- and 364-node behaviour trees, including visible-node ratio, card area, information fields, target dimming, overview coverage and update cost?
- RQ3:** Can topology caching reduce behaviour-tree tick cost across tree sizes and actor counts without changing execution results or becoming unsafe when a resource is mutated in place?

RQ2 deliberately asks about measurable display properties rather than claiming improved usability. Whether the techniques reduce user completion time, error rate or cogni-

tive load requires participant data. The corresponding study is designed in this dissertation, but no participant outcomes are fabricated.

## 1.4 OBJECTIVES AND SUCCESS CRITERIA

The project objectives are to:

1. implement serialisable tree, node, decorator and typed-blackboard resources;
2. implement correct `SUCCESS`, `FAILURE` and `RUNNING` semantics for root, sequence, selector, reactive selector, random selector, parallel, repeat, wait, action and condition behaviour;
3. allow a reusable component to assign a tree and actor to an NPC, with leaf nodes calling actor methods;
4. provide visual creation, connection, disconnection, drag movement, typed editing, validation, save/load, undo and redo;
5. show the current path, node status, blackboard and failure reason only inside the Godot editor;
6. implement large-tree presentation methods behind independent persisted switches and safe reset paths;
7. create deterministic trees and a sufficiently complex 2D scenario to test real decision changes;
8. evaluate correctness, visual geometry, visual regressions, runtime performance and clean installation with reproducible evidence.

Success requires more than launching the plugin. Every runtime node must have focused semantic tests; editor operations must survive save/reload and history operations; the game scenario must demonstrate detection, pursuit, attack, search, retreat and recovery; visual features must be checked with a real renderer; and cached and uncached execution must return identical terminal states. Logs are treated as failed if they contain script errors, leaks, native crashes or illegal memory access, even if an assertion counter reports success.

## 1.5 CONTRIBUTIONS

The main contributions of this work are:

- a distributable Godot 4.6 behaviour-tree plugin with executable composite, leaf and decorator semantics, actor-method integration and a typed blackboard;

- an editor-only live-debug bridge that reports active paths, blackboard values and failure causes without coupling debugging presentation to the running game;
- an integrated set of 19 independently switchable display features, including compact and semantic representations, focus+context, subtree reduction, navigation, path emphasis, accessibility encodings and alternative edge rendering;
- a reproducible evaluation corpus containing deterministic 31-, 121- and 364-node resources, GPU screenshots, raw CSV measurements, formula workbooks and a participant-study protocol;
- a topology cache whose exact signature detects same-size reordering, reparenting, decorator reassignment and resource-instance replacement, together with measurements over nine size/load combinations.

## 1.6 DISSERTATION STRUCTURE

Chapter 2 critically reviews behaviour trees and scalable hierarchy visualisation. Chapter 3 defines the design-science and experimental methodology. Chapter 4 describes the resource model, runtime, editor, display mechanisms, game scenario and test infrastructure. Chapter 5 reports correctness, geometric, visual and runtime results. Chapter 6 interprets those results against the research questions and literature, including limitations. Chapter 7 summarises contributions and future work. Appendices provide feature, test and reproduction references.

# LITERATURE REVIEW

## 2

### 2.1 BEHAVIOUR TREES AS EXECUTABLE HIERARCHIES

A behaviour tree repeatedly evaluates a rooted hierarchy and propagates one of three statuses: `SUCCESS`, `FAILURE` or `RUNNING`. Sequences normally stop at the first failing or running child; selectors stop at the first successful or running child. Leaves inspect state or perform actions. This small status interface permits composition and makes interruption policies explicit (Colledanchise and Ögren, 2018). In games, the same model supports readable high-level choices while actor code retains movement, animation, sensing and combat details.

The simplicity of the three-status interface hides important implementation choices. A running sequence may remember its current child, whereas a reactive selector may re-evaluate high-priority conditions and interrupt a lower-priority running branch. A random selector needs stable selection while running if it is to avoid changing action every frame. Parallel nodes require policies defining how many children must succeed or fail. Repeat and wait nodes require memory that is cleared when the tree restarts or a branch is interrupted. Consequently, editor appearance cannot be separated completely from runtime semantics: child order, decorator ownership and node identity affect execution and must survive serialisation.

Blackboards provide shared keyed state between sensing, conditions and actions. Unreal Engine’s behaviour-tree workflow illustrates the value of combining a blackboard with visual decorators and editor-time runtime observation (Epic Games, 2026). Yet reproducing every feature of a commercial system is neither necessary nor desirable for this project. The relevant baseline is a coherent system that can express meaningful decisions, invoke project-specific actor methods, validate keys and reveal why execution selected or rejected a branch.

### 2.2 NODE-LINK TREES AND SCALABILITY

Node-link diagrams remain appropriate for behaviour trees because edges expose ancestry and sibling groups directly. Classical tidy-tree algorithms aim to produce compact, unambiguous layouts with consistent levels and non-overlapping subtrees (Reingold and Tilford, 1981). SpaceTree later combined node-link structure with dynamic layout, zooming and selective expansion, and its evaluation showed that representation and interaction

jointly affect topology and revisit tasks (Plaisant et al., 2002). These systems establish that automatic placement is not merely cosmetic; it supports the user's spatial model of hierarchy.

The limitation is scale. Node cards consume area, and explicit edges become visually dense as breadth and depth increase. Song et al. compared traditional, list and multi-column views for medium and large fan-outs and found the multi-column interface faster for browsing and understanding the tested hierarchies, as well as preferred overall (Song et al., 2010). This finding is relevant to selectors and sequences with many alternatives, but direct transfer requires care. Their hierarchy ordered children alphabetically; a behaviour tree's horizontal order can define execution priority. A multi-column behaviour-tree layout must therefore display an unambiguous order and must not silently alter the resource.

Bae and Watson examined large layered graphs and developed Quilts, a representation influenced by matrices (Bae and Watson, 2011). Their work is important here less as a layout to copy than as evidence that path tasks can favour a representation different from a conventional node-link diagram. A behaviour-tree debugger can preserve the full editable tree while adding a compact textual path summary and strong active-path encoding. This hybrid strategy avoids replacing a representation that is useful for authoring while targeting a specific runtime-inspection task.

### 2.3 FOCUS, CONTEXT AND OVERVIEW

Furnas formalised generalized fisheye views using a degree of interest (DOI) function: an item's a priori importance is combined with its distance from the current focus to decide its display prominence (Furnas, 1986). Lamping et al. used hyperbolic geometry to place a large hierarchy in a bounded region while magnifying the focus and compressing distant context (Lamping et al., 1995). The attraction for a behaviour-tree editor is clear: a developer could read the node under the pointer without losing all surrounding structure.

Focus+context is not automatically superior. Distortion can move targets and complicate pointing; changing scale can also damage the mental map. Cockburn et al.'s review distinguishes overview+detail, zooming and focus+context, noting that performance depends on task, implementation and transition cost rather than a universal winning technique (Cockburn et al., 2008). The present implementation therefore limits fisheye magnification to 1.20 times, modifies internal card presentation rather than applying an unstable graph transform, compensates around the node centre, and restores all geometry when disabled. It also provides a separate fixed minimap, allowing focus+context and overview+detail to be compared or combined.

Semantic zoom addresses a related but distinct problem. Geometric zoom changes physical size, whereas semantic zoom changes which attributes are presented. For behaviour trees, a low-detail card can retain type colour, icon and short title; a high-detail card can expose method names, parameters, descriptions and decorator conditions. This trades information completeness for density without changing the underlying execution graph.

The distinction matters because earlier versions of this project coupled wheel movement to temporary node scaling, producing shrink-and-grow artefacts. Separating information visibility from graph geometry eliminated that failure mode.

## 2.4 COMPLEXITY MANAGEMENT AND MENTAL-MAP PRESERVATION

When displaying every node is unnecessary, expand-collapse and hide-show operations can reduce complexity. Dogrusoz et al. provide incremental methods and reusable libraries for managing large networks, including expansion, collapse and layout update (Dogrusoz et al., 2018). Their emphasis on incremental change aligns with mental-map research: large unanticipated movement after an edit forces users to relearn locations (Misue et al., 1995). A behaviour-tree collapse operation should therefore preserve the collapsed root, report what has been hidden and avoid moving unrelated branches where possible.

Subtree collapse and subtree focus answer different questions. Collapse replaces descendants with a summary while retaining the rest of the tree; focus isolates a selected branch and its necessary context. Collapse supports incremental simplification, whereas focus is useful when editing a known subsystem. Both operations risk hiding an active node during live debugging. The plugin addresses this with hidden-node summaries, path indicators and explicit expand/focus controls rather than silently deleting view state.

Search highlighting and inactive-branch dimming are softer forms of complexity management. They preserve geometry but reduce visual salience. This supports Shneiderman’s overview-first, zoom-and-filter, details-on-demand principle (Shneiderman, 1996), although the tree remains editable throughout. Shape/icon coding and a colourblind-safe palette add redundant channels so that type is not communicated by colour alone.

## 2.5 RUNTIME EXPLANATION AND FAILURE LOCALISATION

Visual debugging adds time to a static hierarchy. Highlighting only a leaf is insufficient because developers need to know how execution reached it. Conversely, colouring every recently visited node can produce an ambiguous history rather than a current path. The implementation therefore records an ordered root-to-leaf path, per-node status and a single current failure explanation. Non-active branches may be dimmed, while a separate path-summary row remains readable even when the graph is zoomed out.

Decorators make explanation particularly important. A child action can be correct yet never execute because a blackboard comparison, cooldown or time limit blocks the branch. Displaying decorator badges on their owner node and attaching a failure reason connects runtime evidence to the authoring representation. A live blackboard table further exposes key, runtime type, value and schema status. This corresponds to the task-oriented lesson from layered-graph research: optimise the representation for the question the user is asking,

which during debugging is often “why did this branch not run?” rather than “what is the entire topology?”

## 2.6 RESEARCH GAP AND DESIGN IMPLICATIONS

The literature supports several individual mechanisms but does not prescribe their integration in an executable Godot behaviour-tree editor. Four implications guide the project:

1. retain node–link topology and explicit sibling order as the authoring baseline;
2. separate geometric magnification, semantic detail and structural filtering so each can be disabled and evaluated independently;
3. preserve spatial context during collapse and layout, and provide overview or path representations when detail is hidden;
4. evaluate geometric effects and software correctness separately from human-task performance.

The last implication defines the central evidential boundary of this dissertation. Reduced area or fewer visible nodes demonstrates that a representation is more compact, not that it is easier to understand. A participant study is required before making the latter claim.

# RESEARCH METHODOLOGY

## 3.1 RESEARCH STRATEGY

This study uses a design-science and software-engineering strategy. The artefact is a Godot 4.6 plugin and executable demo; knowledge is produced by connecting design requirements from behaviour-tree semantics and visualisation literature to an implemented system, then measuring correctness, display properties and runtime cost. Iterative development was appropriate because editor interactions exposed faults that were not visible in isolated algorithms, including coordinate-space errors, stale transforms, duplicated edges and partially written live-debug snapshots.

The process had five stages. First, supervisor requirements and the dissertation guidance were converted into acceptance criteria. Second, a minimal executable behaviour-tree core and game integration were established. Third, editor operations and live debugging were added. Fourth, visual techniques were implemented behind independent switches, with a disable/reset path tested after each addition. Fifth, deterministic workloads and layered tests were used to compare the completed artefact against its own baseline. Later tasks were followed by full regression because a change to graph rebuilding or resource copying could invalidate an earlier feature.

## 3.2 EXPERIMENTAL FACTORS AND DATASETS

Three deterministic generated trees contain 31, 121 and 364 nodes. They represent a small tree that is comfortably editable, a medium tree exceeding the approximately 50-node readability concern raised during supervision, and a large stress case. Each tree is generated by the same structural rule so that scale, rather than unrelated content, is the main difference. Fixed target nodes allow search and live-debug screenshots to be repeated.

The display experiment uses the conventional full-detail node-link graph as the baseline. The principal conditions are compact cards, semantic zoom, search highlight, subtree focus, subtree collapse, fisheye and enhanced minimap. Additional switch-cycle checks cover all 19 features. Measurements are defined as follows:

- *visible ratio*: visible nodes divided by total nodes;
- *card area*: sum of visible card minimum-width multiplied by minimum-height;
- *information fields*: representative fields retained at the active semantic-detail level;

- *dimmed nodes*: nodes whose salience is reduced by search or runtime-path focus;
- *focus scale gain*: internal magnification of the fisheye target;
- *overview coverage*: nodes represented in the minimap divided by total nodes;
- *operation time*: elapsed time for the measured rebuild, update or switch cycle on the test machine.

These are instrumented interface measures, not psychological measures. They identify how the implementation changes information density and context. The planned human study instead records task completion time, incorrect selections, zoom and pan counts, task success, and seven-point readability and cognitive-load ratings.

Runtime profiling crosses three tree sizes with 1, 10 and 50 runner instances and cache enabled/disabled. Every combination is warmed up and measured seven times. Tick budgets are adjusted so each condition performs a comparable bounded workload. The median mean-tick time is used because it is less sensitive than the arithmetic mean to transient scheduling delays. Correctness assertions require cached and uncached conditions to return the same terminal status.

### 3.3 SOFTWARE CORRECTNESS AND INTEGRATION EVALUATION

Testing is divided into five core layers. Resource/runtime tests validate structure, node semantics, decorators, actor calls, blackboard schemas, debug snapshots and cache invalidation. Editor GUI tests instantiate the plugin interface and exercise creation, dragging, connections, typed fields, save/reload, undo/redo and feature switches. Basic game integration tests verify player and enemy interaction. Complex-arena tests verify detection, chase, directional attack, loss of target, retreat and healing. Real-graphics processing unit (GPU) regression tests capture 1600 by 900 pixel views and assert visible graph properties that cannot be read from Godot's dummy renderer.

The pass policy is deliberately strict. A non-zero process exit, `FAIL:`, `ERROR:`, `SCRIPT ERROR:`, leaked object, resources left in use, access violation or native crash invalidates a suite. Warnings are examined by meaning: a deliberately missing actor method is a safety test and must return `FAILURE`, while an editor file-scan warning after `-quit-after` is expected shutdown behaviour.

### 3.4 VISUAL VALIDATION

Automated assertions cannot detect every presentation fault. The visual protocol therefore renders stable scenes on an NVIDIA GeForce RTX 5070 Laptop GPU using Godot's OpenGL 3.3 Compatibility renderer. Core screenshots cover baseline, type encoding, grayscale, colourblind palette, single connections, compact cards, fisheye enabled/disabled, collapse, search, runtime failure, live blackboard, schema editor, schema-key picker, orthogonal

edges, bundled edges and the complex tree. Four additional controlled-study images cover 121- and 364-node overviews, a distant search target and a deterministic live path.

Visual inspection checks card overlap, port position, edge duplication, text clipping, stale scaling, viewport centring, active-path continuity, dimming reset and minimap coverage. This was essential in diagnosing a fisheye implementation that appeared correct by state variables but used an off-centre transform and interacted incorrectly with wheel zoom.

### 3.5 HUMAN-STUDY DESIGN

The proposed within-participant study compares Baseline, Compact, Collapse, Fisheye, Search and Minimap. Participants complete three task types: locating a named action, interpreting a fixed seven-node runtime path, and editing a known decorator. Conditions are counterbalanced to reduce order effects. The prepared workbook supports 15 participants and 18 trials each, producing 270 trial rows with formula-driven summaries.

The intended recruitment range is 8–15 participants with basic familiarity with games or node editors. Participants receive a short standardised training task. No personally identifying data are required beyond an anonymous identifier and an optional experience category. The final study must follow University ethics and supervisor guidance before recruitment. Because that process has not occurred, no inferential human-performance result appears in this draft.

### 3.6 VALIDITY AND REPRODUCIBILITY

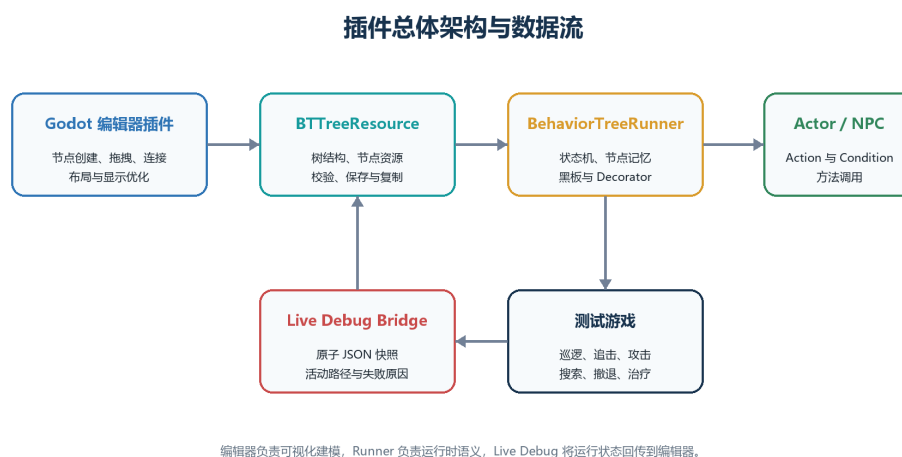
Internal validity is strengthened by fixed resources, deterministic targets, independent switches, repeated profile samples and full regression after integration. Construct validity remains limited because visible area and field count are proxies for density rather than readability. External validity is limited to one Windows laptop, Godot 4.6, generated tree structures and the implemented 2D game. Timing values are therefore reported with hardware and workload rather than presented as universal constants.

Reproducibility materials include raw CSV files, screenshots, test scripts, formula workbooks, a clean-install packaging script and an MIT-licensed versioned plugin archive. Human-study materials are provided separately from automated evidence so that uncollected participant data cannot be mistaken for results.

# SYSTEM DESIGN AND IMPLEMENTATION

## 4.1 ARCHITECTURE AND PROJECT BOUNDARY

The implementation is divided into editor, resource, runtime and validation layers (figure 4.1). The active demonstration project is separated from a synchronised distributable plugin template. Game scenes depend only on serialised resources and the runtime component; editor controls and live-debug presentation are not included in the game graphical user interface (GUI). This boundary prevents an authoring tool from becoming a hidden dependency of exported agent behaviour.



**FIGURE 4.1:** Implemented plugin architecture and data flow between the Godot editor, behaviour-tree resource, runtime runner and actor.

BTreeResource owns a root identifier, an array of node resources and an optional blackboard schema. BTreeNodeResource stores the type, title, description, parameters, graph position, ordinary parent and decorator owner. BehaviorTreeRunner executes a tree against an actor and dictionary blackboard. BehaviorTreeComponent exposes tree, actor path, tick mode and lifecycle as a reusable scene component. In the editor, BTEditorView coordinates the palette, toolbar, graph, inspector, schema editor, history and live-debug panels; BTGraphEdit handles canvas interaction and custom connections; and BTGraphNode renders the card and status.

## 4.2 RESOURCE MODEL AND VALIDATION

Ordinary hierarchy is represented by `parent_id`. Decorators instead use `decorator_parent_id`, preventing them from becoming independent control-flow children. Child queries sort by graph X position, breaking ties by Y position. This makes the visible left-to-right sequence match execution order. Reordering a card therefore changes priority intentionally, while serialisation preserves the designer's layout.

Tree validation rejects missing roots, duplicate identifiers, invalid parent links, illegal decorator ownership and node-specific structural violations. A root must not have a parent; leaves cannot own ordinary children; a decorator must own a valid ordinary node; and required parameters must be present. Saving invokes the same validation used by tests, reducing the chance that an apparently valid graph fails only at runtime.

The blackboard schema contains typed entries for Boolean, integer, floating-point, string and `Vector2` values, together with defaults and a dynamic-key policy. Strict mode reports empty references and keys not declared by the schema. The Inspector presents declared keys in a typed picker while retaining free input for migration and dynamic use. A reverse index identifies which nodes reference a key and which declarations remain unused. Schema editing is integrated into tree-level undo/redo by deep-copying both nodes and schema entries.

## 4.3 RUNTIME SEMANTICS

Every tick returns `SUCCESS`, `FAILURE` or `RUNNING`. Node memory is indexed by node identifier and cleared on restart, tree replacement or branch interruption. The implemented semantics are summarised in table 4.1.

Action and actor-mode condition methods use a common signature: `method(blackboard, delta, node)`. The actor may return a tree status or a Boolean where appropriate. Missing methods fail safely and produce a debug reason rather than raising a fatal script error. Blackboard conditions support existence, non-existence, truth, falsity, equality, inequality and ordered numeric comparisons.

Decorators execute before or around their owner. Implemented modes include blackboard comparison, cooldown, time limit, invert, force result and repeat behaviour. Cooldown memory prevents immediate re-entry; time limit fails a running child after a threshold; invert exchanges success and failure; and force-result wrappers preserve running but replace the terminal result. Resetting a subtree clears decorator and child memory together.

## 4.4 EDITOR AUTHORIZING WORKFLOW

The plugin registers a bottom editor panel. A palette supports click or drag creation. Nodes can be moved, connected from a bottom square port to a top square port, disconnected through edge interaction, duplicated and removed. Composite nodes accept multiple chil-

**TABLE 4.1:** Implemented runtime node semantics.

| Node              | Behaviour                                                                               |
|-------------------|-----------------------------------------------------------------------------------------|
| Root              | Executes its single ordinary child.                                                     |
| Sequence          | Executes ordered children until one fails or runs; remembers a running index.           |
| Selector          | Executes ordered children until one succeeds or runs; remembers a running index.        |
| Reactive Selector | Restarts priority checks each tick and interrupts a lower-priority running branch.      |
| Random Selector   | Chooses a child using a seedable generator and retains the choice while running.        |
| Parallel          | Ticks children and applies configurable success and failure thresholds.                 |
| Repeat            | Repeats a child a fixed number of times or indefinitely, preserving count across ticks. |
| Wait              | Accumulates delta time and succeeds after the configured duration.                      |
| Condition         | Calls an actor predicate or compares a blackboard value.                                |
| Action            | Calls an actor method and converts Boolean or status results into tree status.          |

dren. Context menus expose common operations and subtree display commands. All data-changing commands push a tree snapshot so undo and redo cover creation, deletion, movement, connection, parameter edits, schema edits and collapse state.

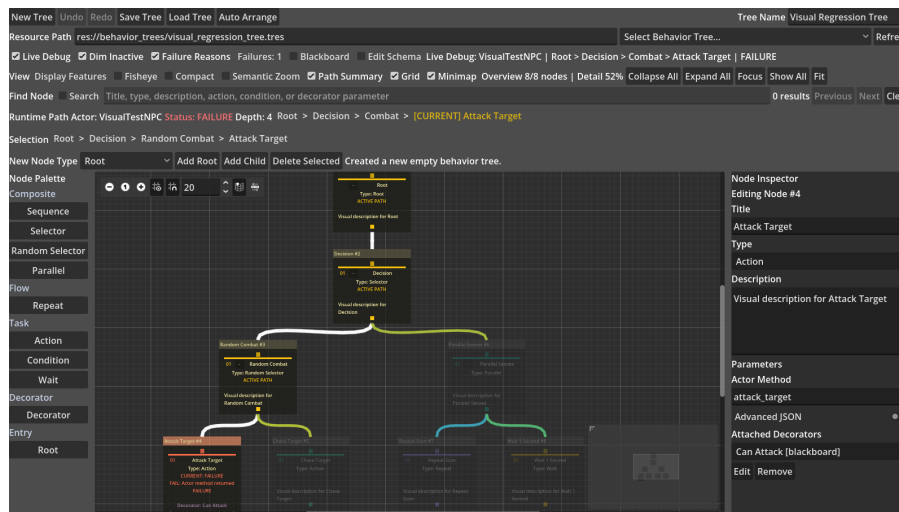
The inspector uses node-specific typed controls rather than requiring hand-written JSON for common fields. Actions choose a method name; conditions choose actor or blackboard mode, key, operator and comparison value; random, parallel, repeat and wait nodes expose their policies; decorators expose mode-specific parameters. An advanced JSON entry remains available for forward compatibility. Invalid JSON is parsed without emitting repeated red console errors, and the previous valid parameter object is retained until the input is repaired.

Connections required special handling because Godot's native `GraphEdit` curve uses side-oriented slots, whereas the intended behaviour-tree presentation uses vertical parent and child ports. In single-connection mode, the native persistent connection layer is hidden and a custom top-to-bottom edge is drawn. The native layer is temporarily restored during drag preview. Custom segment hit-testing preserves right-click disconnection and undo/redraw. Disabling the feature restores native rendering, establishing a safe fallback.

## 4.5 LIVE DEBUGGING AND BLACKBOARD INSPECTION

The runner writes a compact snapshot containing actor identity, tree resource path, active node, ordered path, node statuses, failure reasons, blackboard values and schema information. A process-specific temporary file is atomically published to `res://.godot/behavior_tree_runtime_debug.json`. This avoids the editor reading a half-written JSON object. If a snapshot is truncated, the editor retains the previous complete state and recovers on the next publication.

The editor filters snapshots by the open tree resource and displays an active path, current leaf, path-summary row, failure menu and live blackboard table. Graph cards use status borders and annotations; non-active branches can be dimmed to alpha 0.24 while the active path remains at 1.0. The long runtime path and the current selection are separate scrollable rows, preventing overlap at the 1600 by 900 validation resolution. No corresponding overlay is present in the running game.



**FIGURE 4.2:** Editor-only live debugging with the active path and a failure explanation attached to the relevant branch.

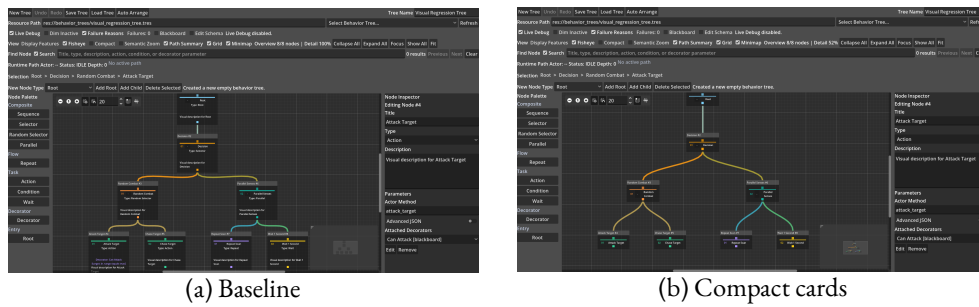
## 4.6 LARGE-TREE DISPLAY MECHANISMS

Nineteen feature definitions are registered in a common switch system. State is persisted in a Godot `ConfigFile`; applying or disabling a feature calls a common refresh path. Switch combinations are tested by enabling features, rebuilding, disabling all features and checking for residue. The mechanisms are grouped below by purpose.

### 4.6.1 INFORMATION-DENSITY CONTROL

Compact cards reduce card width and height while retaining type encoding and a short title. Semantic zoom retains the same graph geometry but changes visible fields according to zoom level: low detail shows identity; higher detail reveals parameters, descriptions

and decorator conditions. Type shape/icon encoding supplements colour, and an optional colourblind-safe palette provides redundant accessible cues.



**FIGURE 4.3:** Full-detail baseline and compact-card presentation of the same behaviour tree.

#### 4.6.2 FOCUS AND STRUCTURAL REDUCTION

Fisheye magnifies the card under the pointer and nearby cards up to 1.20 times while retaining distant context. It changes internal size and font tokens rather than applying `GraphNode.scale`; centre compensation prevents the focus drifting to the right, and disable explicitly restores all presentation values. Subtree collapse hides descendants but reports their count and summaries from the next two levels. Subtree focus isolates a selected branch. Stable incremental layout attempts to reuse valid positions when rebuilding so unrelated branches do not jump.

#### 4.6.3 NAVIGATION AND OVERVIEW

Search matches title, type, description, method and decorator parameters; non-matches are dimmed and `F3/Shift+F3` navigate results. Breadcrumbs expose selected-node ancestors. Fit-to-view computes a scale sufficient for the complete graph, including a 0.10 minimum for stress trees. An enhanced 230 by 150 pixel minimap represents all nodes and the current viewport. Multi-column layout balances wide sibling groups while adding order cues so execution priority remains visible.

#### 4.6.4 EDGE AND EXECUTION REPRESENTATIONS

Orthogonal routing replaces curves with right-angle segments. Edge bundling allows connections with a common direction to share a trunk, reducing repeated parallel geometry. Active-path highlighting, inactive-branch dimming, decorator badges, path summaries and failure annotations add execution-specific information without modifying the resource. Each of these remains optional because highlighting and edge styles can interact differently with tree density and user preference.

## 4.7 RUNTIME TOPOLOGY CACHE

Uncached runtime queries repeatedly scan the node array to find an identifier, gather a parent's children and gather an owner's decorators. The optional cache builds three indexes: node ID to resource, parent ID to ordered children, and owner ID to ordered decorators. Dynamic condition and action results are never cached.

A naïve cache that checks only resource instance and node count is unsafe when a tree is edited in place without changing its size. The final implementation computes an exact topology signature once per external tick. It contains root ID, node count, every node resource instance, node ID, parent ID, decorator owner and X/Y ordering position. A changed signature rebuilds all indexes. Recursive execution reuses the already validated cache for the remainder of that tick. This preserves mutation safety at the cost of an  $O(n)$  validation scan, a deliberate correctness–performance trade-off.

## 4.8 DEMONSTRATION GAME

The complex 2D arena provides behavioural evidence beyond synthetic trees. The player can move, perform melee attacks, dash, heal and briefly cloak. Three guards run behaviour-tree components. At low health a guard retreats and heals; otherwise directional attack branches check range, orientation, cooldown and time limit; a chase branch approaches a visible target; a search branch moves towards the last known location after cloaking; and a patrol branch uses random selection, parallel execution, repeat and wait. Detection uses hysteresis: a visible player is acquired within 330 pixels and released beyond 460 pixels, preventing rapid oscillation at a single threshold.

The game intentionally uses simple generated shapes and texture changes rather than animation-heavy art. This keeps the experimental emphasis on decision logic and makes automated state assertions reliable.

# 5 RESULTS AND ANALYSIS

## 5.1 FUNCTIONAL CORRECTNESS

The final core regression contained 447 assertions and all passed (table 5.1). The runtime/resource suite was the largest source of semantic evidence, while editor GUI and GPU suites covered interactions and presentation. A 180-physics-frame arena smoke test retained three active runners and a live complex enemy. Editor launch logs for the active project, distribution template and clean-install project contained no real failure, script error, leak, crash or illegal memory access.

**TABLE 5.1:** Final core automated regression.

| Suite                | Passed | Total | Principal coverage                                                                               |
|----------------------|--------|-------|--------------------------------------------------------------------------------------------------|
| Resource and runtime | 153    | 153   | Structure, node semantics, decorators, actor calls, schemas, debug bridge and cache invalidation |
| Editor GUI           | 185    | 185   | Graph editing, history, typed controls, schema tools, live blackboard and 19 display switches    |
| Basic game           | 13     | 13    | Three enemies, movement, damage, patrol, death/restart and runner lifecycle                      |
| Complex arena        | 26     | 26    | Detection, pursuit, directional attack, search, retreat, recovery and new composite nodes        |
| Real-GPU visual      | 70     | 70    | Seventeen 1600 by 900 screenshots and visual invariants                                          |
| Total                | 447    | 447   | All core assertions passed                                                                       |

Additional suites were kept separate to avoid inflating the core total. The controlled human-study fixture passed 21 of 21 GPU assertions, runtime profiling and topology safety passed 511 of 511 assertions, and packaging passed 53 of 53 checks. The version 0.9.0 archive contained only the plugin directory, included an MIT licence and SHA-256 manifest, and enabled successfully in an otherwise empty Godot 4.6 project.

The complex-arena tests showed that agents were genuinely driven by tree execution rather than a parallel hard-coded controller. Entering detection range set blackboard target

state and selected chase; entering oriented attack range selected the corresponding attack branch; cloaking caused target loss and last-known-position search; and low health selected retreat and healing. Action methods still performed movement and damage, but branch choice, ordering and interruption came from the tree resource.

## 5.2 DISPLAY-DENSITY AND CONTEXT RESULTS

Table 5.2 reports the principal measurements. Compact cards retained every node while reducing total card area by 55.9% at all three sizes because the card dimensions were reduced consistently. Semantic zoom reduced representative fields from six to one at low detail, an 83.3% reduction, while preserving node geometry. These results demonstrate density changes, not task performance.

**TABLE 5.2:** Principal display measurements on fixed generated trees.

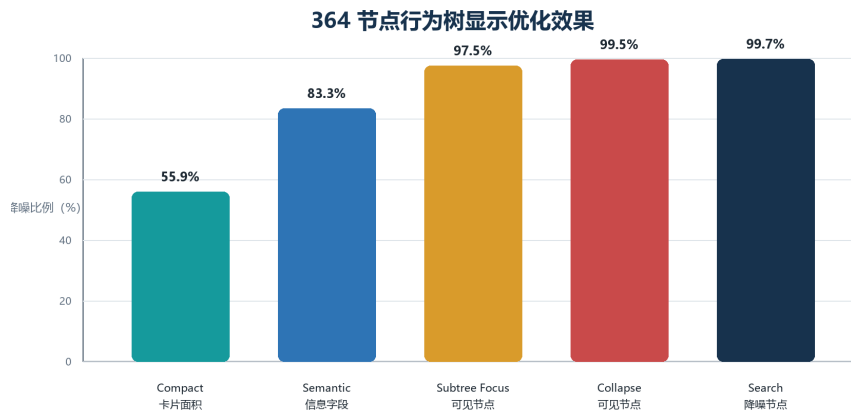
| Measure                               | 31 nodes  | 121 nodes | 364 nodes  |
|---------------------------------------|-----------|-----------|------------|
| Baseline card area (px <sup>2</sup> ) | 1,162,500 | 4,537,500 | 13,650,000 |
| Compact card area (px <sup>2</sup> )  | 512,864   | 2,001,824 | 6,022,016  |
| Compact area reduction                | 55.9%     | 55.9%     | 55.9%      |
| Subtree-focus visible nodes           | 7         | 8         | 9          |
| Focus visible-node reduction          | 77.4%     | 93.4%     | 97.5%      |
| Subtree-collapse visible nodes        | 2         | 2         | 2          |
| Collapse visible-node reduction       | 93.5%     | 98.3%     | 99.5%      |
| Search-dimmed nodes                   | 30        | 120       | 363        |
| Fisheye target gain                   | 1.20x     | 1.20x     | 1.20x      |
| Minimap coverage                      | 100%      | 100%      | 100%       |

Subtree focus became proportionally more selective as tree size increased: it showed seven of 31 nodes, eight of 121 and nine of 364. Collapse was more aggressive, retaining two visible entry cards in all three fixtures. On the largest tree, these correspond to 97.5% and 99.5% reductions respectively. Search preserved the entire layout but dimmed every node except the fixed target, reaching 99.7% non-target dimming for 364 nodes. The fixed minimap represented all nodes at every scale.

The fisheye result is intentionally modest. The target increased by 20%, enough to make local content more prominent without the severe relocation associated with stronger distortion. Unlike collapse or focus, fisheye did not reduce visible-node count. It is therefore complementary rather than directly comparable on that metric.

## 5.3 RUNTIME-PATH AND OVERVIEW RESULTS

Inactive-branch dimming maintained active alpha 1.0 and inactive alpha 0.24, a 4.17:1 contrast. The dimmed proportions were 83.9%, 97.5% and 99.2% for the three trees. First



**FIGURE 5.1:** Measured geometric and salience effects for principal methods on the 364-node fixture.

application rose from 1.902 ms to 45.037 ms as the number of cards increased, but steady state remained between 0.39662 and 0.43618 ms (table 5.3). This suggests that applying state to many cards is the scaling cost, while a no-change update is inexpensive.

**TABLE 5.3:** Active-branch dimming and enhanced-minimap measurements.

| Nodes | Dimmed | First dim (ms) | Steady dim (ms) | Map coverage | Map steady (ms) |
|-------|--------|----------------|-----------------|--------------|-----------------|
| 31    | 26     | 1.902          | 0.40548         | 100%         | 0.00132         |
| 121   | 118    | 6.256          | 0.43618         | 100%         | 0.00140         |
| 364   | 361    | 45.037         | 0.39662         | 100%         | 0.00174         |

The minimap remained 230 by 150 pixels and represented 31, 121 and 364 nodes. Toggle-cycle time increased from 9.3163 ms to 126.79382 ms, while steady status update remained below 0.002 ms. This separates construction cost from ongoing runtime observation. GPU screenshots confirmed that viewport framing and node markers remained within the minimap and that disabling the feature removed it cleanly.

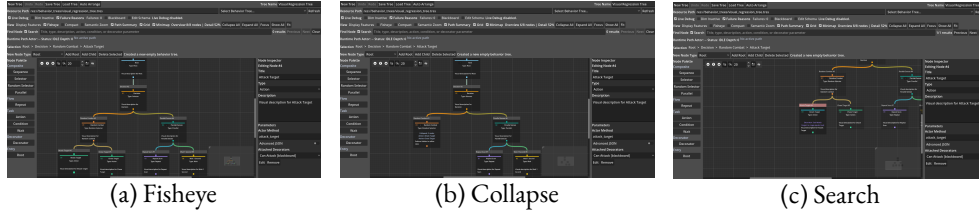
## 5.4 VISUAL-REGRESSION FINDINGS

Visual testing identified defects that were not adequately represented by numeric metrics. Earlier fisheye code transformed whole graph nodes, causing wheel zoom to make all cards appear to shrink and later grow, leaving stale scale after disable and positioning the visual focus to the right of the pointer. Replacing graph transforms with internal size and font changes, using centre compensation and resetting presentation state removed all three symptoms. The disable screenshot matched baseline geometry rather than retaining transformed cards.

Search initially selected a distant card in the inspector but failed to centre it at low zoom because graph-space coordinates were mixed with scaled scroll offsets. Converting through

the active `GraphEdit.zoom` fixed the viewport calculation, and a distant-target regression now checks that the card enters the visible region. Dynamic per-layer height also removed overlap in a 36-node tree containing multiple decorator badges.

Other verified findings include one persistent edge per relationship rather than a native/custom double edge, readable top and bottom square ports, no overlap between runtime-path and selection rows, no stale dimming after reset, and correct display of decorator summaries, schema-key choices and blackboard type state. Figure 5.2 illustrates three distinct mechanisms.

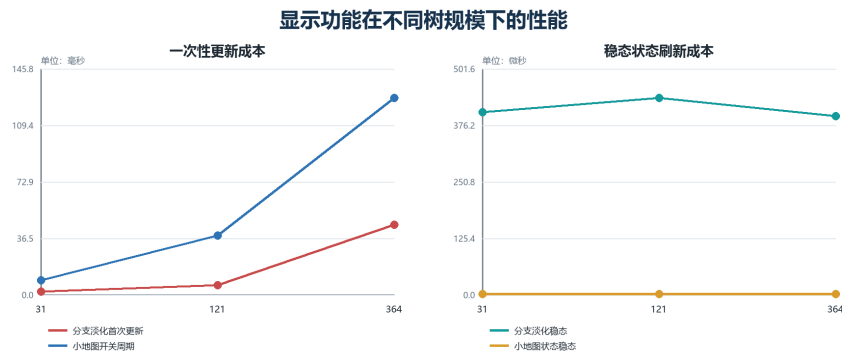


**FIGURE 5.2:** Examples of focus+context, structural reduction and salience-based navigation in the implemented editor.

## 5.5 RUNTIME-CACHE RESULTS

All cached and uncached profiles reached the same `SUCCESS` state. Safety tests additionally mutated trees without changing their node count: sibling order, parent relationship, decorator owner and same-ID resource instance were changed independently. Every mutation invalidated and rebuilt the cache. Whole-tree replacement and disabled-cache fallback also passed.

Table 5.4 reports median per-tick values across seven samples. The safe cache reduced median tick cost for every tested condition. Reduction increased strongly with tree size because uncached recursive queries repeatedly scan a larger array, whereas indexed lookup offsets the single signature scan. The 31-node case achieved 15.98–26.46%; 121 nodes achieved 52.61–56.02%; and 364 nodes achieved 76.65–78.00%.



**FIGURE 5.3:** Runtime tick scaling with the safe topology cache enabled and disabled.

**TABLE 5.4:** Median runtime profile by tree size and runner count.

| Nodes | Runners | No cache ( $\mu\text{s}/\text{tick}$ ) | Cache ( $\mu\text{s}/\text{tick}$ ) | Reduction |
|-------|---------|----------------------------------------|-------------------------------------|-----------|
| 31    | 1       | 314.834                                | 264.522                             | 15.98%    |
| 31    | 10      | 376.313                                | 276.741                             | 26.46%    |
| 31    | 50      | 372.556                                | 294.096                             | 21.06%    |
| 121   | 1       | 2405.729                               | 1140.190                            | 52.61%    |
| 121   | 10      | 2429.358                               | 1068.500                            | 56.02%    |
| 121   | 50      | 2687.255                               | 1270.075                            | 52.74%    |
| 364   | 1       | 15963.829                              | 3727.207                            | 76.65%    |
| 364   | 10      | 15926.812                              | 3635.463                            | 77.17%    |
| 364   | 50      | 15318.540                              | 3370.560                            | 78.00%    |

The data do not show a clear monotonic penalty from adding runners because the benchmark fixes total tick work approximately rather than running every actor for a fixed wall-clock duration. The valid interpretation is therefore within each size/load pair: cached and uncached conditions execute comparable work on the same machine. Cross-machine or production-frame claims require a different benchmark.

## 5.6 SUMMARY AGAINST RESEARCH QUESTIONS

RQ1 is supported by the complete test matrix and complex game: the plugin authors, serialises, validates, executes and debugs non-trivial agent behaviour. RQ2 is answered at the instrumented-display level: compact, semantic, structural, salience and overview methods produced distinct measurable effects and passed real-renderer reset checks. Human readability remains unanswered. RQ3 is supported for the tested workload: exact-signature topology caching preserved semantics and substantially reduced median tick cost, especially on larger trees.

# DISCUSSION

## 6.1 A COMPLETE SYSTEM AS A RESEARCH INSTRUMENT

The first research question concerned feasibility. The completed plugin demonstrates that Godot's general graph and editor APIs are sufficient for a behaviour-tree workflow approaching the functional baseline expected from established tools: serialisable resources, reusable actor components, ordered composites, conditions, actions, decorators, blackboards and live execution inspection. The complex arena is important evidence because a graph that only renders correctly could still be disconnected from agent behaviour. Detection, chase, directional attack, search and recovery were selected by the resource at runtime, while actor methods provided physical effects.

Functional breadth also exposed why a behaviour-tree editor is more than a generic tree visualiser. Moving a sibling changes priority; hiding a decorator can conceal the reason a task did not run; changing tree resources must clear execution memory; and live state is updated while the graph is being inspected. The architecture's separation of resource, runner and editor reduced coupling, but the integration surface remained large. The 447 core assertions should therefore be interpreted as evidence over specified cases, not proof of absence of all defects.

The decision to exclude service nodes was appropriate to the project's scope. Parallel, random, repeat and wait behaviour, typed blackboards and decorators already permit a sufficiently complex demonstration. Implementing another Unreal-inspired abstraction would have added runtime and editor combinations without directly addressing the main research question. A graduation project benefits more from a defensible, tested boundary than from an incomplete attempt to reproduce a commercial engine.

## 6.2 INTERPRETING THE VISUALISATION RESULTS

The display techniques address different tasks and their percentages are not a common utility scale. Collapse's 99.5% visible-node reduction is valuable when a developer knows which subtrees can be abstracted, but it also removes topology and can hide an unexpected runtime branch. Fisheye's 20% magnification seems numerically smaller, yet it preserves the entire graph and may better support local reading during navigation. Search's 99.7% dimming is powerful for a known textual target but provides little help when the developer does not know a node's name. Minimap's 100% coverage provides orientation but not

readable details.

This task dependence agrees with the literature. Focus+context techniques balance local detail and global structure (Furnas, 1986; Lamping et al., 1995), whereas overview+detail offers a stable global reference at the cost of divided attention (Cockburn et al., 2008). Song et al.’s multi-column advantage applies specifically to large fan-out browsing (Song et al., 2010); it does not establish that every behaviour tree should be reflowed. Quilts improved particular path tasks in large layered graphs (Bae and Watson, 2011), motivating the plugin’s path summary rather than wholesale replacement of the editable node–link view.

The strongest design outcome is therefore not one winning technique but controlled composition. A developer can use compact cards to increase density, minimap to retain overview, search to locate a target and semantic detail to inspect it. During runtime, branch dimming and a textual path can replace search. Independent switches also made fault isolation practical: when fisheye caused stale transforms, it could be disabled without affecting collapse or live debugging.

Mental-map preservation remains only partially addressed. Stable incremental layout and centre-compensated fisheye reduce unnecessary movement, and collapse retains a visible owner and summary. Nevertheless, automatic layout can still move cards when a previously invalid position conflicts with a new subtree. Dogrusoz et al. and Misue et al. emphasise that spatial continuity is a central property of interactive graph management (Dogrusoz et al., 2018; Misue et al., 1995). A future evaluation should measure whether users can revisit a node after repeated collapse/expand cycles rather than relying only on overlap counts.

### 6.3 LIVE DEBUGGING AS A READABILITY MECHANISM

Runtime inspection changes the large-tree problem from general topology reading to temporal explanation. The path-summary view, active edges, inactive dimming, failure annotation and live blackboard provide redundant evidence: graph colour communicates location, text communicates order, and the blackboard communicates decision inputs. This is especially useful for decorators because the visible failure can be attached to the owner rather than inferred from a child that never ran.

The use of an atomically published snapshot was an engineering response to the editor and game running as separate processes. It is simple and inspectable, but file polling is not ideal for high-frequency traces or remote targets. It can also report only the most recent merged state rather than a complete time series. A socket or Godot debugger protocol extension could support event history and multiple remote actors, but would increase complexity and require stronger synchronisation. For this project, robust latest-state inspection was the more appropriate trade-off.

## 6.4 RUNTIME-CACHE TRADE-OFFS

The performance experiment answers RQ3 positively for the fixed workloads. Improvement grows with tree size because indexed lookup replaces repeated linear scans inside recursive execution. The exact topology signature avoids a common optimisation error: assuming that resource identity and node count imply unchanged structure. Tests demonstrate correct invalidation for changes that preserve count but alter execution.

The cache is not free. Every external tick computes an  $O(n)$  signature, reducing the potential gain on a 31-node tree. This explains why the small-tree range is 15.98–26.46%, much lower than the 364-node range. An event-driven generation counter could make validation  $O(1)$ , but only if every mutating path reliably increments it. Godot resources may be edited directly, so a generation counter alone would weaken safety. A hybrid strategy could use editor change signals during authoring and periodic exact validation at runtime.

The benchmark must also be interpreted carefully. Mean tick microseconds include GDScript execution and test-harness overhead on one Windows laptop. Runner-count conditions use bounded total work, so they do not estimate the exact frame cost of 50 production agents all ticking continuously. The results establish a relative effect under controlled pairs, not a universal throughput guarantee.

## 6.5 RELIABILITY AND PACKAGING

The strict log policy found issues that assertion-only testing could have missed, including asynchronous timer objects surviving scene shutdown and a native console crash triggered by piping Godot output through a PowerShell native pipeline. Replacing scene-tree timers with node-local state machines removed lifecycle leakage; invoking Godot directly with an absolute `-log-file` avoided the console-pipeline failure. These are relevant outcomes because a plugin used for debugging must not itself introduce misleading shutdown errors.

The clean-install package test strengthens reproducibility. It verifies not only that source files exist but that the ZIP boundary is correct, manifest hashes match and Godot can import and enable the plugin in an empty project. The remaining release work is social and compatibility-oriented: a final icon, Asset Library description and testing against explicitly selected Godot versions.

## 6.6 THREATS TO VALIDITY

### 6.6.1 CONSTRUCT VALIDITY

Card area, visible ratio, field count and dimmed count measure presentation properties, not comprehension. A smaller view may omit information needed for a task; stronger dimming may reduce awareness of alternatives. The dissertation therefore does not claim that a 55.9% area reduction equals a 55.9% readability improvement. The planned participant measures are required to test the usability hypothesis.

### 6.6.2 INTERNAL VALIDITY

Generated trees share a deterministic pattern, which improves comparability but can make search and layout easier than irregular authored trees. Timing samples can be affected by OS scheduling and GPU/editor warm-up. Seven runtime samples and median aggregation reduce but do not eliminate this variability. Visual screenshots validate stable fixtures, not every possible feature combination.

### 6.6.3 EXTERNAL VALIDITY

The system was tested on Godot 4.6, Windows and one NVIDIA GPU. Godot's internal GraphEdit layers can change between versions, especially because single-connection rendering hides a native child layer. Results may not transfer directly to mobile editors, different DPI scales or trees with thousands of nodes. The demonstration focuses on 2D combat AI; non-game robotics uses may require different tick and safety policies.

### 6.6.4 RESEARCHER AND PARTICIPANT VALIDITY

The developer designed both artefact and automated metrics, creating a risk that tests favour implemented mechanisms. Raw data and explicit limits improve auditability, but independent participant evaluation is still needed. No participant data have yet been collected, so there is no risk of overstating a small or biased sample in the current results; equally, there is no empirical basis for a human-usability conclusion.

## 6.7 PRACTICAL IMPLICATIONS

For small trees, full-detail node-link presentation remains reasonable and has the lowest learning cost. For medium and large trees, the results support offering techniques by task rather than forcing a single adaptive mode: compact/semantic detail for overview, collapse/focus for subsystem editing, search for known targets, minimap for orientation, and active-path/failure tools for runtime diagnosis. Safe defaults should favour stable geometry; more distorting techniques should be opt-in and easily reset.

For plugin authors, the work demonstrates that display innovation must be tested together with authoring semantics. A visually elegant edge is insufficient if it cannot be clicked to disconnect; a collapse function is incomplete if new children attach invisibly without a summary; and a runtime highlight is unsafe if a partial debug message clears valid state. The integration tests are therefore part of the contribution, not merely post-implementation quality assurance.

# 7 CONCLUSIONS

## 7.1 SUMMARY

This dissertation investigated how a complete Godot 4.6 visual behaviour-tree system can support real game-agent execution while making large trees more inspectable. The work began from two connected problems: Godot provides general graph-editor primitives rather than an integrated behaviour-tree workflow, and conventional full-detail node-link views become crowded as the number of nodes, parameters and runtime annotations grows.

The resulting version 0.9.0 plugin implements serialisable trees, typed blackboards, decorators, ordered and reactive composites, random and parallel control, repeat and wait memory, actor-method actions and conditions, reusable NPC components, undo/redo and editor-only live debugging. A complex 2D scenario demonstrates detection, pursuit, directional attack, last-known-position search, retreat and healing under tree control.

The first research question is answered positively for the defined scope: Godot's editor and resource APIs supported a practical authoring, execution and debugging pipeline. The second is answered in terms of instrumented display effects. Compact cards reduced measured card area by 55.9%; semantic zoom reduced representative fields by 83.3%; focus and collapse reduced visible nodes by up to 97.5% and 99.5%; search dimmed 99.7% of non-target nodes; and the enhanced minimap retained 100% node coverage. Real-GPU regression verified that these effects could be enabled and disabled without the previously observed fisheye, centring, overlap or double-edge defects. The third research question is also answered positively for the tested workload: exact-signature caching preserved execution results and reduced median tick cost by 15.98–78.00%, with larger gains on larger trees.

## 7.2 CONTRIBUTIONS

The project contributes a usable Godot plugin rather than a disconnected visual prototype; an integrated catalogue of switchable large-tree and runtime-inspection mechanisms; a safe topology-cache design; and a reproducible evidence package containing 447 passing core assertions, 21 controlled visual checks, 511 profile assertions, 53 package checks, raw measurements and fixed screenshots. It also contributes a negative but important methodological result: geometric compression must not be reported as proven human readability. The prepared participant protocol makes that boundary explicit.

### 7.3 LIMITATIONS

The current evidence is limited to Godot 4.6 on one Windows/GPU environment and deterministic trees no larger than 364 nodes. Automated results establish implementation properties but not long-term authoring productivity or cognitive load. The debug bridge exposes latest state rather than a full timeline. Service nodes, subtree resource reuse, Asset Library distribution and a cross-version compatibility matrix are outside the completed scope.

### 7.4 FUTURE WORK

The immediate priority is to conduct the prepared counterbalanced study with 8–15 participants. Baseline, Compact, Collapse, Fisheye, Search and Minimap should be compared on target location, path interpretation and decorator editing. Completion time, errors, pan/zoom actions, success and subjective ratings should be analysed without merging geometrically incomparable conditions into a single unsupported score.

Further engineering work should add reusable subtree assets and explicit subtree interfaces, a debug timeline with pause/step controls, actor selection for multiple simultaneous runners, and an event-assisted topology generation counter backed by periodic exact validation. Visual work could examine animated incremental transitions and user-configurable DOI functions while measuring revisit performance and mental-map retention. Finally, release work should validate selected Godot versions and prepare an Asset Library listing.

### 7.5 FINAL STATEMENT

The project shows that a graduation-scale implementation can move beyond imitating an existing editor. By combining executable semantics, editor-only explanation, switchable focus and complexity controls, strict regression testing and explicit evidential limits, the plugin provides both a practical AI authoring tool and a defensible platform for continued research into readable visual programming.

# BIBLIOGRAPHY

- Colledanchise, M. and Ögren, P. (2018). *Behavior Trees in Robotics and AI: An Introduction*. CRC Press. doi:10.1201/9780429489105.
- Furnas, G. W. (1986). Generalized fisheye views. *Proceedings of CHI '86*, 16–23. doi:10.1145/22339.22342.
- Lamping, J., Rao, R. and Pirolli, P. (1995). A focus+context technique based on hyperbolic geometry for visualizing large hierarchies. *Proceedings of CHI '95*, 401–408. doi:10.1145/223904.223956.
- Cockburn, A., Karlson, A. and Bederson, B. B. (2008). A review of overview+detail, zooming, and focus+context interfaces. *ACM Computing Surveys*, 41(1), 1–31. doi:10.1145/1456650.1456652.
- Song, H., Kim, B., Lee, B. and Seo, J. (2010). A comparative evaluation on tree visualization methods for hierarchical structures with large fan-outs. *Proceedings of CHI 2010*, 223–232. doi:10.1145/1753326.1753359.
- Bae, J. and Watson, B. (2011). Developing and evaluating Quilts for the depiction of large layered graphs. *IEEE Transactions on Visualization and Computer Graphics*, 17(12), 2268–2277. doi:10.1109/TVCG.2011.187.
- Dogrusoz, U., Karacelik, A., Safarli, I., Balci, H., Dervishi, L. and Siper, M. C. (2018). Efficient methods and readily customizable libraries for managing complexity of large networks. *PLOS ONE*, 13(5), e0197238. doi:10.1371/journal.pone.0197238.
- Misue, K., Eades, P., Lai, W. and Sugiyama, K. (1995). Layout adjustment and the mental map. *Journal of Visual Languages and Computing*, 6(2), 183–210. doi:10.1006/jvlc.1995.1010.
- Reingold, E. M. and Tilford, J. S. (1981). Tidier drawings of trees. *IEEE Transactions on Software Engineering*, SE-7(2), 223–228. doi:10.1109/TSE.1981.234519.
- Plaisant, C., Grosjean, J. and Bederson, B. B. (2002). SpaceTree: Supporting exploration in large node link tree, design evolution and empirical evaluation. *Proceedings of IEEE InfoVis 2002*, 57–64. doi:10.1109/INFVIS.2002.1173148.

- Shneiderman, B. (1996). The eyes have it: A task by data type taxonomy for information visualizations. *Proceedings of IEEE Visual Languages 1996*, 336–343. doi:10.1109/VL.1996.545307.
- Godot Engine contributors (2026a). GraphEdit class reference. [https://docs.godotengine.org/en/stable/classes/class\\_graphedit.html](https://docs.godotengine.org/en/stable/classes/class_graphedit.html). Accessed 9 August 2026.
- Godot Engine contributors (2026b). EditorPlugin class reference. [https://docs.godotengine.org/en/stable/classes/class\\_editorplugin.html](https://docs.godotengine.org/en/stable/classes/class_editorplugin.html). Accessed 9 August 2026.
- Epic Games (2026). Behavior Tree in Unreal Engine: Overview. <https://dev.epicgames.com/documentation/en-us/unreal-engine/behavior-tree-in-unreal-engine---overview>. Accessed 9 August 2026.

# FEATURE REFERENCE

**TABLE A.1:** Independent display features in the final editor.

| Feature                    | Default | Purpose                                                         |
|----------------------------|---------|-----------------------------------------------------------------|
| Fisheye / Focus+Context    | On      | Magnifies the pointer neighbourhood up to 1.20x.                |
| Subtree Collapse / Expand  | On      | Hides descendants and presents a count plus two-level summary.  |
| Compact Mode               | Off     | Reduces cards to type encoding and short title.                 |
| Active Path Highlight      | On      | Emphasises the current runtime path.                            |
| Non-active Branch Dimming  | Off     | Reduces inactive branch alpha during live debug.                |
| Multi-column Layout        | Off     | Balances wide sibling groups while preserving order cues.       |
| Enhanced Minimap           | On      | Provides a 230 by 150 overview and viewport marker.             |
| Semantic Zoom              | Off     | Changes information detail without transforming graph geometry. |
| Path Summary View          | On      | Displays actor, state, depth and ordered runtime path.          |
| Decorator Condition Badges | On      | Summarises decorator conditions on owner cards.                 |
| Search + Highlight         | On      | Searches node and decorator text and dims non-matches.          |
| Orthogonal Edges           | Off     | Uses right-angle routes.                                        |
| Edge Bundling              | Off     | Shares edge trunks in dense regions.                            |
| Stable Incremental Layout  | Off     | Reuses non-conflicting positions during layout.                 |
| Breadcrumb Navigation      | On      | Shows and navigates selected-node ancestry.                     |
| Failure Reason Annotation  | On      | Displays node-level failure labels and summaries.               |
| Shape / Icon Type Encoding | On      | Supplements colour with geometry and symbols.                   |
| Accessibility Palette      | Off     | Enables colourblind-safer colours and keyboard navigation.      |

| Feature                     |  |         | Default | Purpose                                               |
|-----------------------------|--|---------|---------|-------------------------------------------------------|
| Single Connection Rendering |  | Render- | On      | Displays one custom persistent edge with hit testing. |

# REPRODUCTION AND EVIDENCE

## INDEX

All paths below are relative to the project repository root.

**Active Godot project:** `testgame/testgame/`

**Distributable plugin source:** `visual_scripting/addons/behavior_tree_editor/`

**Complex tree:** `testgame/testgame/behavior_trees/complex_guard_validation_tree.tres`

**Core test scripts:** `testgame/testgame/tests/`

**Raw display data:** `testgame/testgame/test_results/display_optimization_raw.csv`

**Runtime profile:** `testgame/testgame/test_results/runtime_profile.csv`

**Visual screenshots:** `testgame/testgame/test_results/visual/`

**Human-study protocol:** `research/display_optimization/human_comparison_study_protocol.md`

**Evaluation workbook:** `research/display_optimization/behavior_tree_complete_validation_and_display_evaluation.xlsx`

**Versioned package:** `dist/behavior-tree-editor-0.9.0.zip`

The principal test commands are recorded in `AGENTS.md` and in the generated project report. Real visual regression must use `-rendering-method gl_compatibility` rather than `headless dummy` rendering, because the dummy renderer cannot provide the required viewport textures.

# PLANNED PARTICIPANT STUDY

The prepared study contains six display conditions and three tasks per condition. A participant first completes a standardised practice task, then performs 18 counterbalanced trials. The software records condition, tree size, target, completion time, incorrect selections, zoom count, pan count and success. After each condition, the participant gives seven-point readability and cognitive-load ratings. The workbook supports up to 15 participants and 270 rows.

No participant rows have been collected. Before use, the author must obtain any required ethics approval, confirm the protocol with the supervisor, replace prototype instructions with participant-facing wording, and verify that consent and withdrawal procedures meet University requirements.

# DRAFT COMPLETION CHECKLIST

Before final submission, the author should:

1. replace all identity and supervisor placeholders;
2. revise the declaration to match University policy, including permitted AI assistance;
3. verify the current submission deadline and exact word-count rules on Tabula;
4. conduct or formally remove the planned participant study and update Abstract, Results, Discussion and Conclusion consistently;
5. have the supervisor review the literature claims, research questions and evidential limits;
6. run the final text through Turnitin according to departmental guidance;
7. compile with the supplied Warwick template and inspect every page, reference and figure;
8. include the final word count and any required project repository/archive information.